9.6 Regex: Capturing and Backreferences

Lesson 4. Use a captured group again, in the pattern and in a replacement.

Where you are. Lesson 4 of 5. Read note 9.5 first. Next is note 9.7.

This note is how to use a captured group again: inside the pattern, and in a replacement. The summary table is at the end.

Note 9.5 introduces groups. This note is about groups that remember text, and about using that text again.

By the end of this note you should be able to:

1. Tell a capturing group from a non-capturing group.
2. Number groups from the opening parenthesis, including nested groups.
3. Use a named group `(?<name>...)`.
4. Backreference a group inside a pattern, and reuse it in a replacement.

1. The notation

Both kinds of group bundle part of a pattern. The difference is whether the engine remembers what the group matched.

Notation	Type	Remembers the match?
`(...)`	Capturing group	Yes, numbered 1, 2, 3, ...
`(?:...)`	Non-capturing group	No
`(?<name>...)`	Named capturing group	Yes, by name and by number

Python writes a named group as `(?P<name>...)`.

2. What grouping does

A group lets a quantifier or `|` apply to several characters at once.

- `ab+` is `a`, then one or more `b`: `ab`, `abb`, `abbb`.
- `(ab)+` is the pair `ab` repeated: `ab`, `abab`, `ababab`.
- `gr(a|e)y` matches `gray` or `grey`.

Capturing and non-capturing groups both do this. The next sections are about the extra memory.

3. What capturing adds

A capturing group saves its matched text so you can:

1. Extract the piece afterward.
2. Backreference it inside the same pattern (`\\1` in the regex, `"\\1"` in R).
3. Reuse it in a replacement.

4. Numbering

Groups are numbered by the position of the opening parenthesis, left to right. Nested groups count. Non-capturing groups do not.

For `((\\d{4})-(\\d{2}))-(\\d{2})` on `"2026-10-27"`:

- Group 1 is `((\\d{4})-(\\d{2}))`, the text `2026-10`.
- Group 2 is `(\\d{4})`, the year.
- Group 3 is `(\\d{2})`, the month.
- Group 4 is `(\\d{2})`, the day.

```
str_match("2026-10-27", "((\\d{4})-(\\d{2}))-(\\d{2})")
```

```
     [,1]         [,2]      [,3]   [,4] [,5]
[1,] "2026-10-27" "2026-10" "2026" "10" "27"
```

Column 1 is the whole match. Columns 2 through 5 are groups 1 through 4.

5. What a non-capturing group does

`(?:...)` groups for structure and saves nothing. Use it when you need `|` or a quantifier and you do not want the text back.

On `"2026-10-27"`:

Pattern	Groups saved
`(\\d{4})-(\\d{2})-(\\d{2})`	3: `2026`, `10`, `27`
`(?:\\d{4})-(\\d{2})-(\\d{2})`	2: `10`, `27`

```
str_match("2026-10-27", "(\\d{4})-(\\d{2})-(\\d{2})")
```

```
     [,1]         [,2]   [,3] [,4]
[1,] "2026-10-27" "2026" "10" "27"
```

```
str_match("2026-10-27", "(?:\\d{4})-(\\d{2})-(\\d{2})")
```

```
     [,1]         [,2] [,3]
[1,] "2026-10-27" "10" "27"
```

The year is still matched. It is not a column. The month is group 1, and the day is group 2.

That keeps numbering tidy: structural parentheses do not shift `\\1` and `\\2`. Nothing is stored for the group you skipped. A backreference cannot point at a parenthesis you did not capture.

6. The same pattern, two ways

Match `gray` or `grey`, and capture the whole word:

```
str_match("gray", "(gr(?:a|e)y)")
```

```
     [,1]   [,2]
[1,] "gray" "gray"
```

Group 1 is the word. `(?:a|e)` only makes the choice.

```
str_match("gray", "(gr(a|e)y)")
```

```
     [,1]   [,2]   [,3]
[1,] "gray" "gray" "a"
```

Now there are two groups: group 1 is `gray`, and group 2 is `a`. The inner group is clutter when you only wanted the word.

7. Named groups

A name documents the group, and you do not have to count:

```
(?<year>\\d{4})-(?<month>\\d{2})-(?<day>\\d{2})
```

In an R string that is `"(?<year>\\d{4})-(?<month>\\d{2})-(?<day>\\d{2})"`.

The engine still numbers them: `year` is group 1, `month` is group 2, `day` is group 3. Inside the pattern, the named backreference is `\\k<year>`, written `"\\k<year>"` in R.

8. Groups in `str_match()` and `str_replace()`

`str_match()` returns the full match, then one column per capturing group.

```
x <- "2026-10-27"

str_match(x, "(\\d{4})-(\\d{2})-(\\d{2})")
```

```
     [,1]         [,2]   [,3] [,4]
[1,] "2026-10-27" "2026" "10" "27"
```

A non-capturing group is left out:

```
str_match(x, "(?:\\d{4})-(\\d{2})-(\\d{2})")
```

```
     [,1]         [,2] [,3]
[1,] "2026-10-27" "10" "27"
```

In a replacement, `\\1` and `\\2` are the captured text:

```
str_replace("John Smith", "(\\w+) (\\w+)", "\\2, \\1")
```

```
[1] "Smith, John"
```

9. A backreference needs a capturing group

You can only refer back to a capturing group.

```
str_detect(c("bookkeeper", "banana"), "(\\w)\\1")
```

```
[1]  TRUE FALSE
```

`(\\w)\\1` matches a doubled letter. `bookkeeper` has `oo` and `kk`. `banana` has no doubled letter.

```
str_detect("ee", "(?:\\w)\\1")
```

That is an error: `Back-reference to a non-existent capture group`. `(?:\\w)` matched a letter and did not store it, so there is no group 1.

10. Backreference inside the pattern

A backreference means “match this same text again.”

Style	In the regex	In an R string
Numbered	`\\1`, `\\2`	`"\\1"`, `"\\2"`
Named	`\\k<name>`	`"\\k<name>"`

Doubled letter, numbered and named:

```
x <- c("bookkeeper", "banana")

str_detect(x, "(\\w)\\1")
str_detect(x, "(?<ch>\\w)\\k<ch>")
```

```
[1]  TRUE FALSE
[1]  TRUE FALSE
```

A repeated word:

```
y <- "this is is a test"

str_extract(y, "\\b(\\w+)\\s+\\1\\b")
str_extract(y, "\\b(?<w>\\w+)\\s+\\k<w>\\b")
```

```
[1] "is is"
[1] "is is"
```

`\\b` is a word boundary, so the repeat is a whole word. `\\s+` is the space between the two copies.

Matching quotes, where the closer has to be the same mark as the opener:

```
z <- "She said 'hi' and \"bye\""

str_extract_all(z, "(['\"]).*?\\1")
str_extract_all(z, "(?<q>['\"]).*?\\k<q>")
```

```
[[1]]
[1] "'hi'"    "\"bye\""

[[1]]
[1] "'hi'"    "\"bye\""
```

Group 1 (or `q`) is the opening quote. `\\1` requires the same character at the end. `.*?` takes as little as possible in between, so each pair stops at its own closer.

11. Reference in the replacement

In stringr, the replacement uses `\\1`, `\\2`, and so on. This is the update: put the captured text back in a new arrangement.

Reorder a date, `2026-10-27` to `27/10/2026`:

```
d <- "2026-10-27"

str_replace(d, "(\\d{4})-(\\d{2})-(\\d{2})", "\\3/\\2/\\1")
```

```
[1] "27/10/2026"
```

Swap two names:

```
str_replace("John Smith", "(\\w+) (\\w+)", "\\2, \\1")
```

```
[1] "Smith, John"
```

Keep one copy of a duplicated word:

```
str_replace_all("this is is a test", "\\b(\\w+)\\s+\\1\\b", "\\1")
```

```
[1] "this is a test"
```

Wrap every number in brackets:

```
str_replace_all("a1 b22 c333", "(\\d+)", "[\\1]")
```

```
[1] "a[1] b[22] c[333]"
```

`\\0` is the entire match, not a group:

```
str_replace_all("a1 b22", "\\d+", "<\\0>")
```

```
[1] "a<1> b<22>"
```

12. Named groups in the replacement

stringr's replacement understands numbers. A named group is also numbered, so you can name the groups in the pattern and still write `\\1` in the replacement:

```
str_replace(
  d,
  "(?<y>\\d{4})-(?<m>\\d{2})-(?<day>\\d{2})",
  "\\3/\\2/\\1"
)
```

```
[1] "27/10/2026"
```

To use the names in the replacement, call `stringi`. ICU writes a named replacement as `${name}`:

```
stringi::stri_replace_first_regex(
  d,
  "(?<y>\\d{4})-(?<m>\\d{2})-(?<day>\\d{2})",
  "${day}/${m}/${y}"
)
```

```
[1] "27/10/2026"
```

That call works in stringi 1.8.9. The pattern is typed with `\\d`, like every other pattern in this note. The replacement `${day}` has no backslash.

13. Extracting groups

`str_match()` is the reference without an update. With names, stringr 1.6 labels the group columns:

```
m <- str_match(d, "(?<y>\\d{4})-(?<m>\\d{2})-(?<day>\\d{2})")
m
colnames(m)
m[, "y"]
```

```
                 y      m    day
[1,] "2026-10-27" "2026" "10" "27"

[1] ""    "y"   "m"   "day"

[1] "2026"
```

The first column is the whole match, and its name is blank. `"y"`, `"m"`, and `"day"` are groups 1, 2, and 3, so `m[, 2]` and `m[, "y"]` are the same cell.

14. The same idea in other languages

	Numbered	Named
Pattern backreference	`\\1`	JavaScript and PCRE: `\\k<name>`. Python: `(?P=name)`
Replacement	JavaScript: `$1`. Python: `\\1`	JavaScript: `$<name>`. Python: `\g<name>`
Define the group	`(...)`	JavaScript and PCRE: `(?<name>...)`. Python: `(?P<name>...)`

JavaScript:

```
"2026-10-27".replace(/(\d{4})-(\d{2})-(\d{2})/, "$3/$2/$1");
"2026-10-27".replace(/(?<y>\d{4})-(?<m>\d{2})-(?<d>\d{2})/, "$<d>/$<m>/$<y>");
```

Both return `"27/10/2026"`. A JavaScript regex literal does not double the backslash.

Python:

```
import re
re.sub(r"(\d{4})-(\d{2})-(\d{2})", r"\3/\2/\1", "2026-10-27")
re.sub(r"(?P<y>\d{4})-(?P<m>\d{2})-(?P<d>\d{2})", r"\g<d>/\g<m>/\g<y>", "2026-10-27")
```

Both return `'27/10/2026'`. A raw Python string, `r"..."`, keeps one backslash for the regex engine. That is the same role as R's doubled backslash.

15. Summary

You want	Type
Save a piece	`(...)`
Group without saving	`(?:...)`
Name the piece	`(?<name>...)`
That same text again, inside the pattern	`\\1`, or `\\k<name>`
That piece in a replacement	`"\\1"`
The whole match in a replacement	`"\\0"`
Read the saved pieces	`str_match(x, p)`. Column 1 is the whole match. Later columns are the groups, numbered from the opening parenthesis.

A named group is also numbered, so `\\1` and `\\k<name>` can appear in the same pattern. `(?:...)` is skipped in the numbering. Use a name when there are many groups. Use a number for a short pattern.